Project 1

Code:

```
format long;    % Set display format to avoid scientific notation


% Main program for running GA over 10 runs with varying parameters
num_runs = 10;    % Number of runs
num_generations = 2500;    % Fixed number of generations
final_results = zeros(num_runs, 1);    % Store the final best value for each run
all_best_values = zeros(num_generations, num_runs);    % Store best fitness values
for each generation for all runs
params_table = cell(num_runs, 6);    % Initialize a 10x6 cell array to store
parameters and final fitness values


% Load required variables
load('StandardComponentValues.mat');    % Load Res, ThVal, ThBeta, etc.
Tdata = -40:5:85;
Vdata = 1.026e-1 + -1.125e-4 * Tdata + 1.125e-5 * Tdata.^2;


lb = [1 1 1 1 1 1];    % Lower bounds
ub = [70 70 70 70 9 9];    % Upper bounds


% Define constraint function (example)
```

```
function c = nonlcon(x)
    c = x(1) + x(2) - 100;    % Constraint: x(1) + x(2) <= 100
end

% Running GA for 10 runs with different options for each run
for run = 1:num_runs
    % Vary GA parameters for each run, but keep the number of generations fixed
at 2500
    population_size = 200;
    crossover_frac = 0.1;
    mutation_frac = 0.5;

    % Store the parameters for this run in the table
    params_table{run, 1} = population_size;
    params_table{run, 2} = num_generations;    % Fixed number of generations
(2500)
    params_table{run, 3} = crossover_frac;
    params_table{run, 4} = mutation_frac;

    % Set options for this run
    options = struct('PopulationSize', population_size, 'Generations',
num_generations, ...
```

```
        'CrossoverFrac', crossover_frac, 'MutationFrac', mutation_frac);

    % Call myGA function with the current parameters and nonlcon as the
constraint
    [xOpt, fVal, best_values] = myGA(@(x)objectiveFunction(round(x), Res, ThVal,
ThBeta, Tdata, Vdata), ...
        6, [], [], [], [], lb, ub, @nonlcon, [], options);

    % Store the final best value of the current run
    final_results(run) = fVal;

    % Store the best value for each generation for this run (for plotting
convergence)
    all_best_values(:, run) = best_values;    % Save the best value for each
generation

    % Store the convergence value at the 2500th generation in params_table
    params_table{run, 5} = best_values(end);

    % Calculate and store the standard deviation of best values over generations
for this run
    params_table{run, 6} = std(best_values);
```

```
end

% Summarize results: best, worst, average, and standard deviation of fVals over 10
runs
best_result = min(final_results);
worst_result = max(final_results);
mean_result = mean(final_results);
std_result = std(final_results);

% Display the summary table in a clean, aligned format
disp('Summary of 10 Runs:');
fprintf('%-20s %-20s %-20s %-20s\n', 'Best_Result', 'Worst_Result', 'Mean_Result',
'Std_Result');
fprintf('%-20.7f %-20.7f %-20.7f %-20.7f\n', best_result, worst_result, mean_result,
std_result);

% Find the best run and generate the fitted curve
[~, best_run_idx] = min(final_results);    % Find the index of the best run
best_xOpt = round(xOpt);    % Ensure best solution parameters are integers
V_fitted = voltageCurve(Tdata, best_xOpt, Res, ThVal, ThBeta);    % Generate fitted
curve
```

```
% Plot target curve and fitted curve
figure;
plot(Tdata, Vdata, '-*', 'DisplayName', 'Target Curve');  % Target curve
hold on;
plot(Tdata, V_fitted, '-o', 'DisplayName', 'Fitted Curve');  % Fitted curve
legend;
title('Target and Fitted Curve');
xlabel('Temperature (°C)');
ylabel('Voltage (V)');
hold off;

% Calculate and plot convergence trend showing the average of best values from
all runs for each generation
avg_best_values = mean(all_best_values, 2);  % Calculate the average best values
for each generation across runs

% Plot the convergence curve showing the average of best values from all runs
figure;
plot(1:num_generations, avg_best_values);
xlabel('Generation');
ylabel('Average Best Fitness Value');
```

```
title('Convergence Curve of GA (Average of 10 Runs)');



% Genetic Algorithm Function
function [xOpt, fVal, best_values] = myGA(objectiveFunc, nvars, A, b, Aeq, beq, lb,
ub, nonlcon, intCon, options, varargin)
    % Initialize parameters
    pop_size = options.PopulationSize;
    num_generations = options.Generations;
    crossover_rate = options.CrossoverFrac;
    mutation_rate = options.MutationFrac;
    penalty_coefficient = 1000;    % Constraint penalty coefficient
    shrink_value = 0.5;
    SUBPOP = 1;    % Number of subpopulations

    % Define FieldDR for crtrp function (real-value representation)
    FieldDR = [lb; ub];    % Bounds for variables: rows are lower and upper bounds

    % Use crtrp to generate the initial population within the bounds
    population = crtrp(pop_size, FieldDR);
```

```
    % Record the best value for each generation
    best_values = zeros(num_generations, 1);    % Initialize each generation's best value array
    for gen = 1:num_generations
        % Calculate fitness values
        fitness_values = arrayfun(@(i) objectiveFunc(population(i, :), varargin{:}), 1:pop_size)';

        % Constraint penalty
        penalty_values = zeros(pop_size, 1);
        for i = 1:pop_size
            if ~isempty(nonlcon)
                % Call nonlinear constraint function
                constraint_violations = nonlcon(population(i, :));
                % Calculate total constraint violation and apply penalty
                total_violation = sum(max(0, constraint_violations));
                penalty_values(i) = penalty_coefficient * total_violation;
            end
        end
        fitness_values = fitness_values + penalty_values;

        % Selection of parents
```

```
        num_parents = pop_size;
        ranked_fitness = ranking(fitness_values);    % Sort with ranking function
        selected_indices = rws(ranked_fitness, num_parents);    % Select with rws
function
        selected_parents = population(selected_indices, :);

        % Crossover operation
        offspring = xovmp(selected_parents, crossover_rate);    % Multi-point
crossover xovmp

        % If offspring are fewer than the population size, add more individuals
        if size(offspring, 1) < pop_size
            num_additional = pop_size - size(offspring, 1);
            additional_individuals = population(randi([1, pop_size],
num_additional, 1), :);
            offspring = [offspring; additional_individuals];
        end

        % Mutation operation
        MutOpt = [mutation_rate, shrink_value];
        mutated_offspring = mutbga(offspring, FieldDR, MutOpt);
```

```
        % Reinsertion
        InsOpt = [1, 1.0];
        population = reins(population, mutated_offspring, SUBPOP, InsOpt,
fitness_values);

        % Record the best fitness value for the current generation
        best_values(gen) = min(fitness_values);
    end

    % Return final optimal solution
    [fVal, idx] = min(fitness_values);
    xOpt = population(idx, :);
end
```

Output

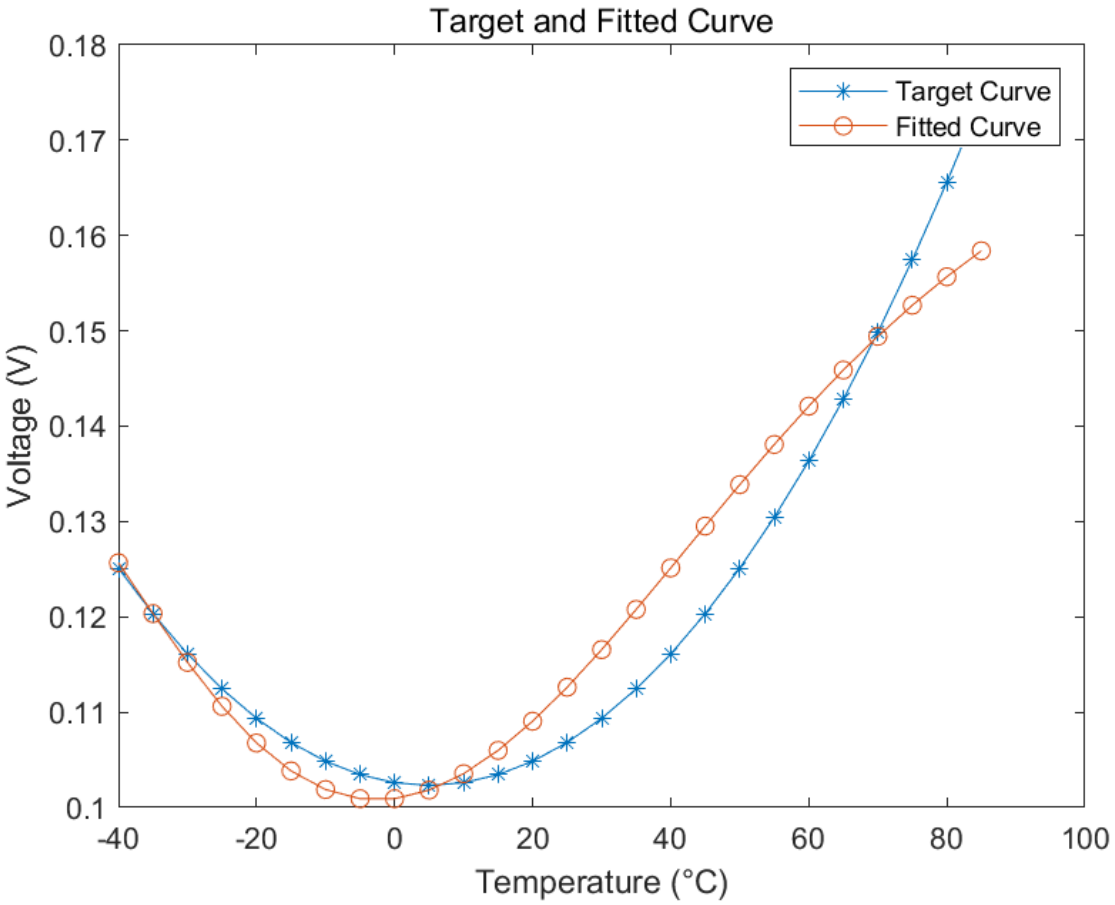
In my project, I finetuned 3 GA parameters. They were population size, crossover fraction and mutation fraction.

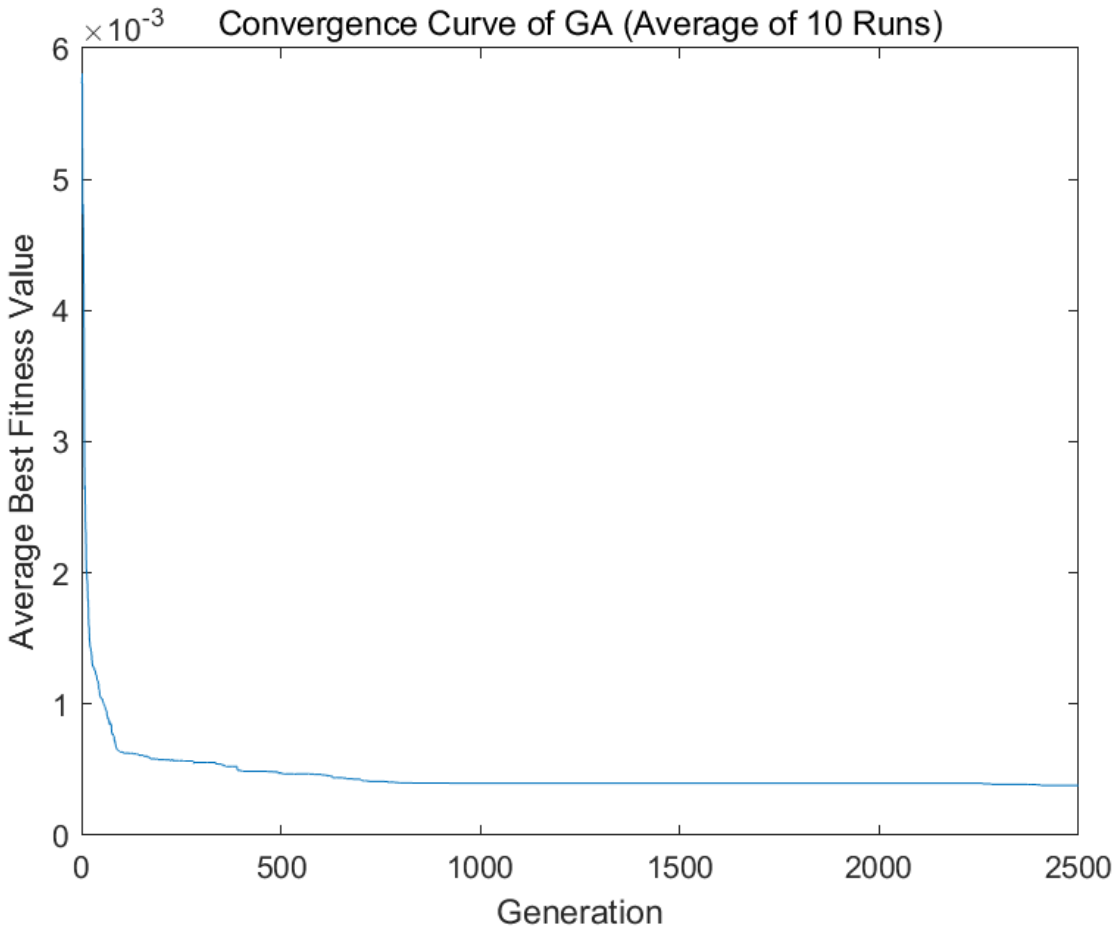
1. population size.

- First setting

- `population_size = 200;`
- `crossover_frac = 0.1;`
- `mutation_frac = 0.1;`

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000668            0.0012440            0.0004779            0.0004728
```

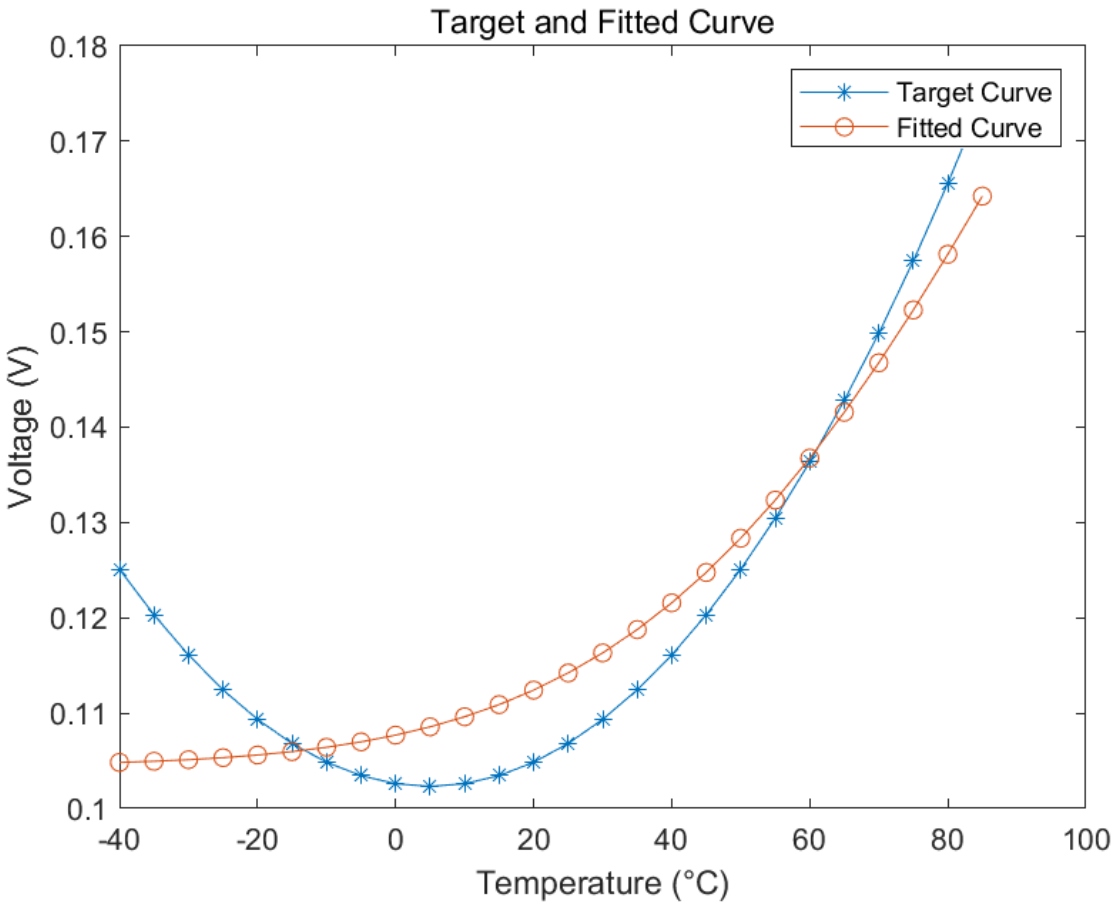


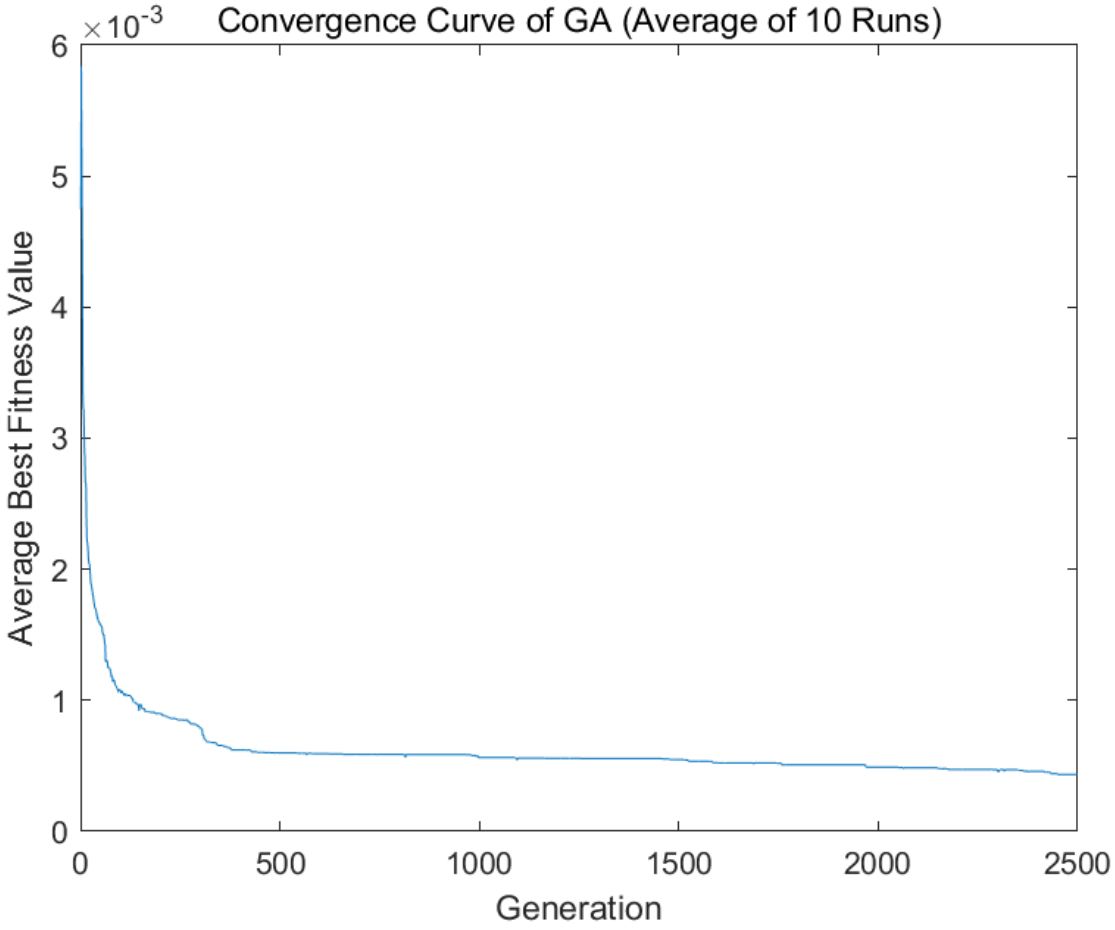


➢ Second setting

- population_size = 300;
- crossover_frac = 0.1;
- mutation_frac = 0.1;

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000442            0.0011719            0.0004344            0.0004457
```

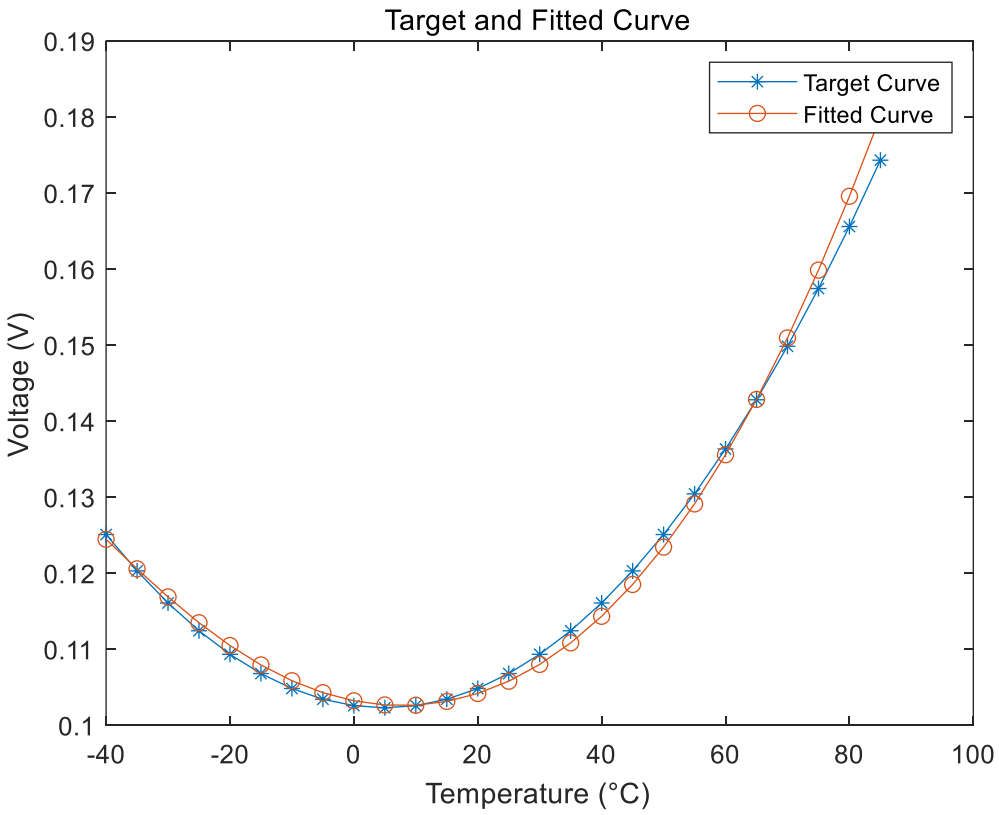

Target and Fitted Curve
Target Curve
Fitted Curve
Voltage (V)
Temperature (°C)
0.18
0.17
0.16
0.15
0.14
0.13
0.12
0.11
0.1
-40
-20
0
20
40
60
80
100

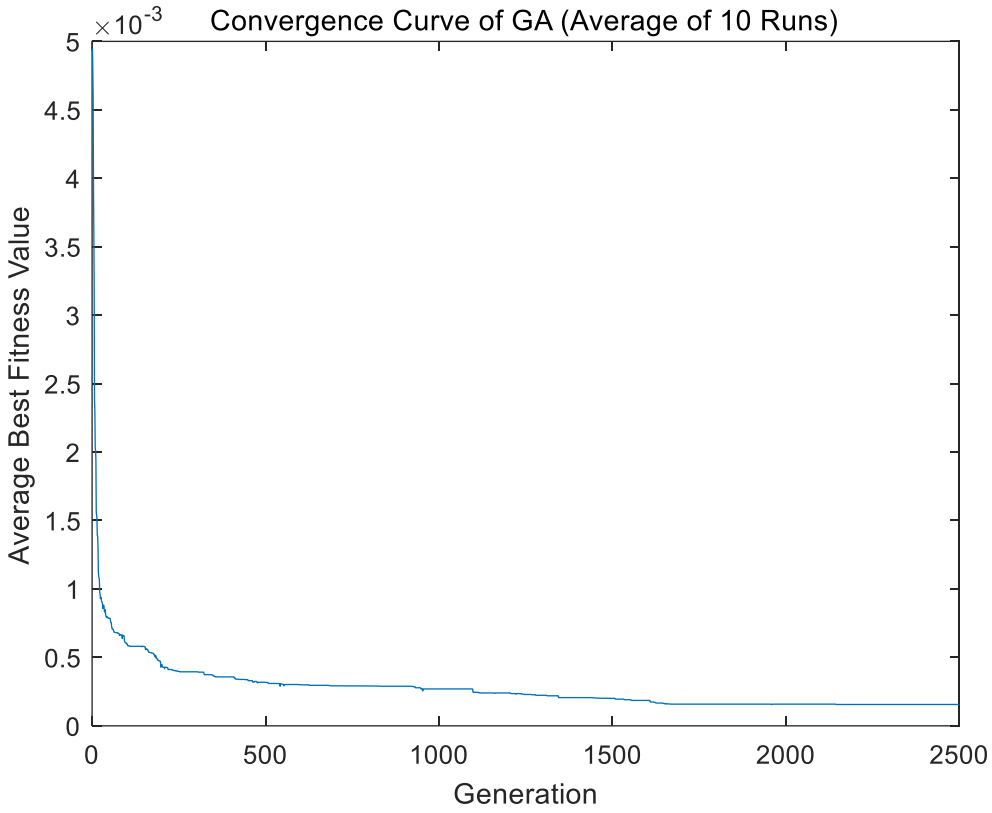

Convergence Curve of GA (Average of 10 Runs)
×10-3
Average Best Fitness Value
Generation
6
5
4
3
2
1
0
0
500
1000
1500
2000
2500

➢ Third setting

- `population_size = 400;`
- `crossover_frac = 0.1;`
- `mutation_frac = 0.1;`

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000294            0.0008350            0.0001559            0.0002429
```

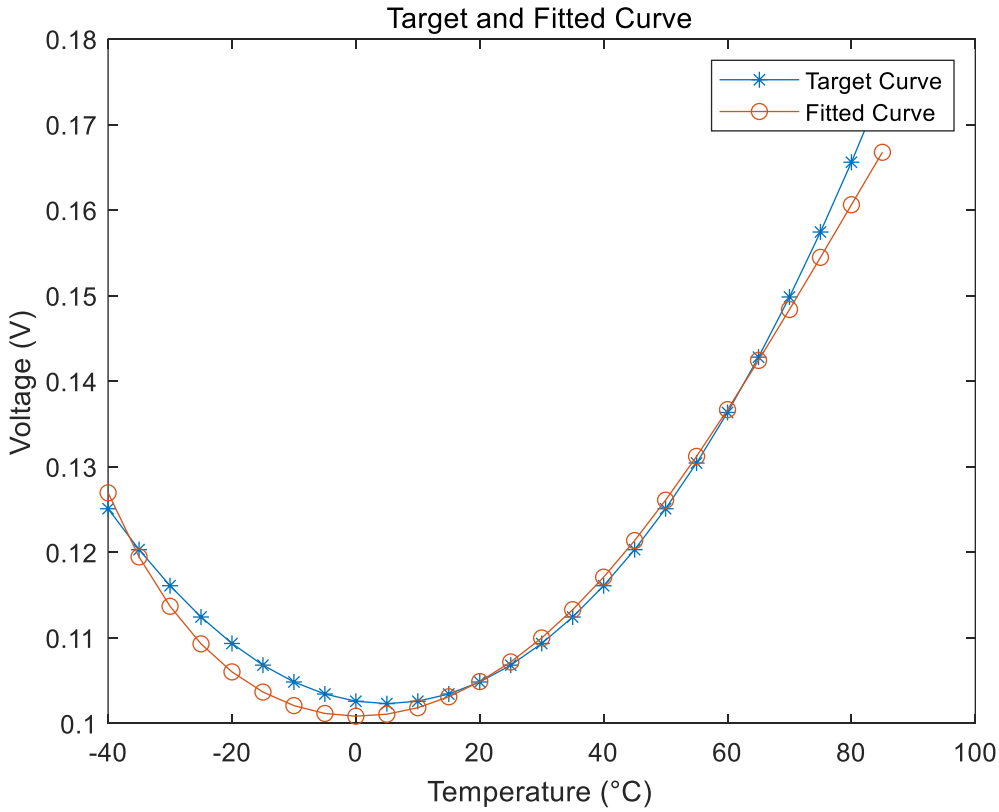


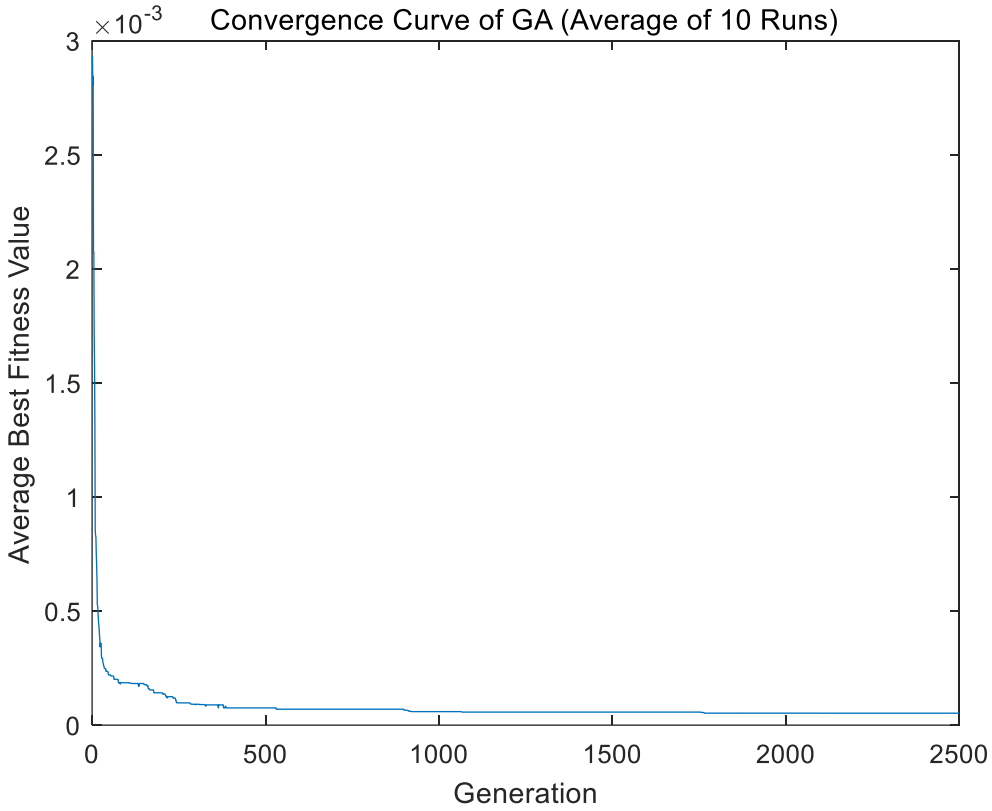


➢ Fourth setting

- `population_size = 1000;`
- `crossover_frac = 0.1;`
- `mutation_frac = 0.1;`

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000233            0.0001357            0.0000523            0.0000336
```





Discussion

In the above four GA setting, I kept increasing the population size, from

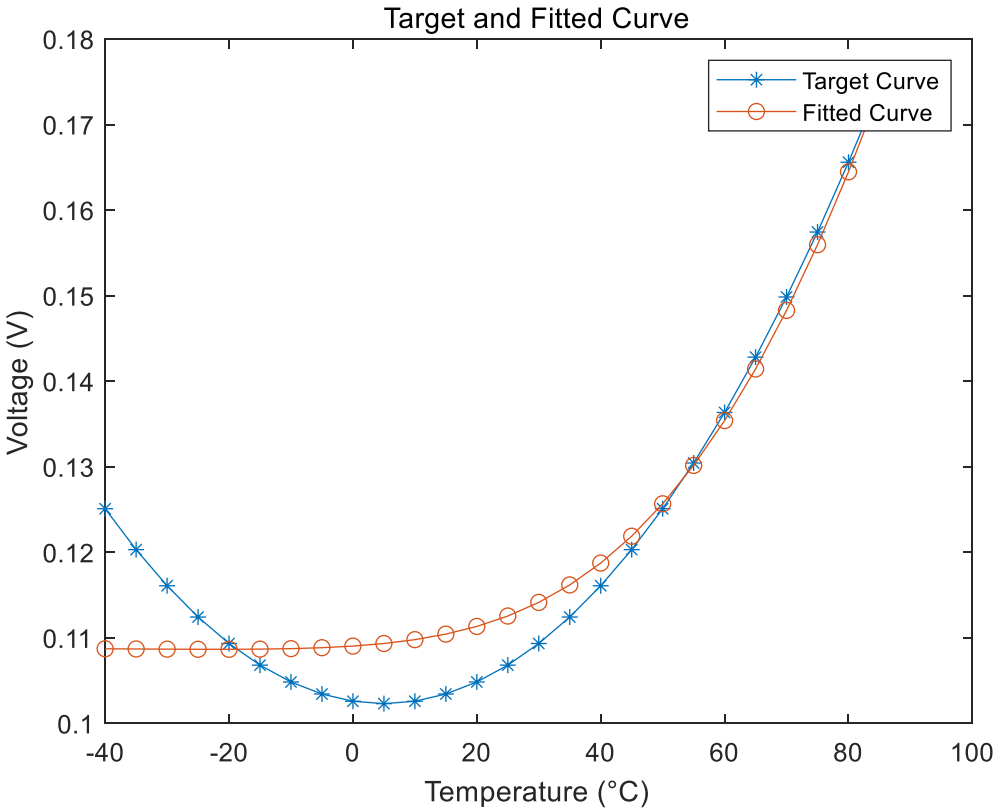
200 to 1,000, and found that the results were gradually approaching the ideal value of zero. The standard deviation was also smaller when population size got bigger. Therefore, I believed that increasing the population size could improve the optimization quality. This might be due to that larger population sizes tended to improve optimization quality by maintaining diversity and reducing the likelihood of premature convergence.

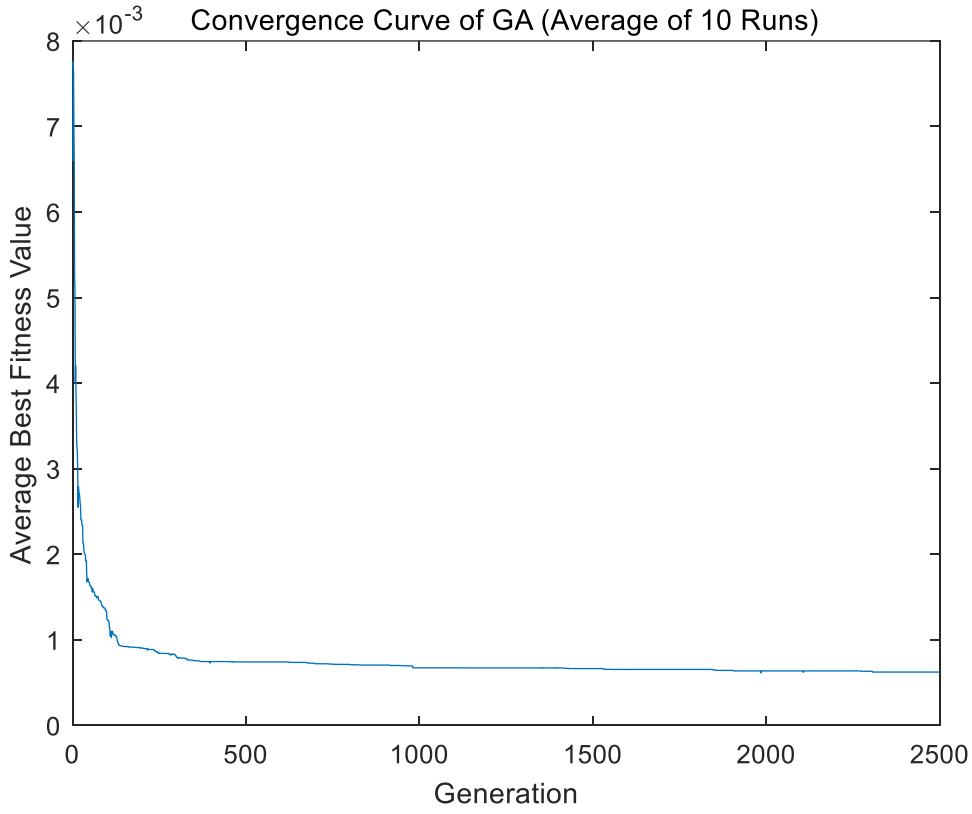
2. crossover fraction

- Fifth setting

- population_size = 200;
- crossover_frac = 0.01;
- mutation_frac = 0.1;

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0001097            0.0011516            0.0006231            0.0004036
```

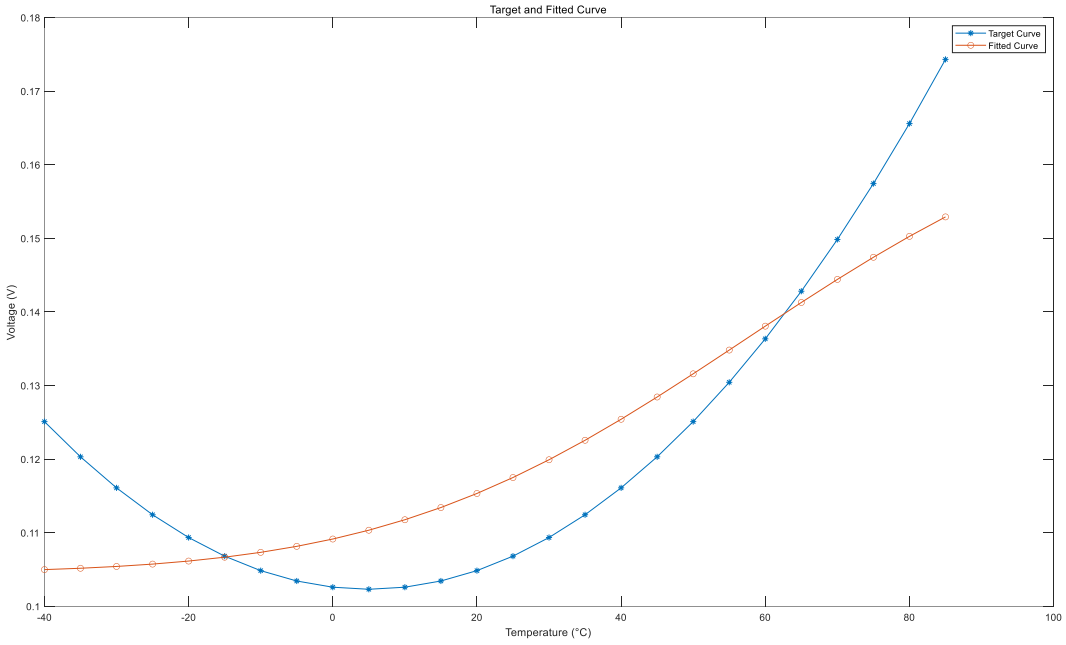


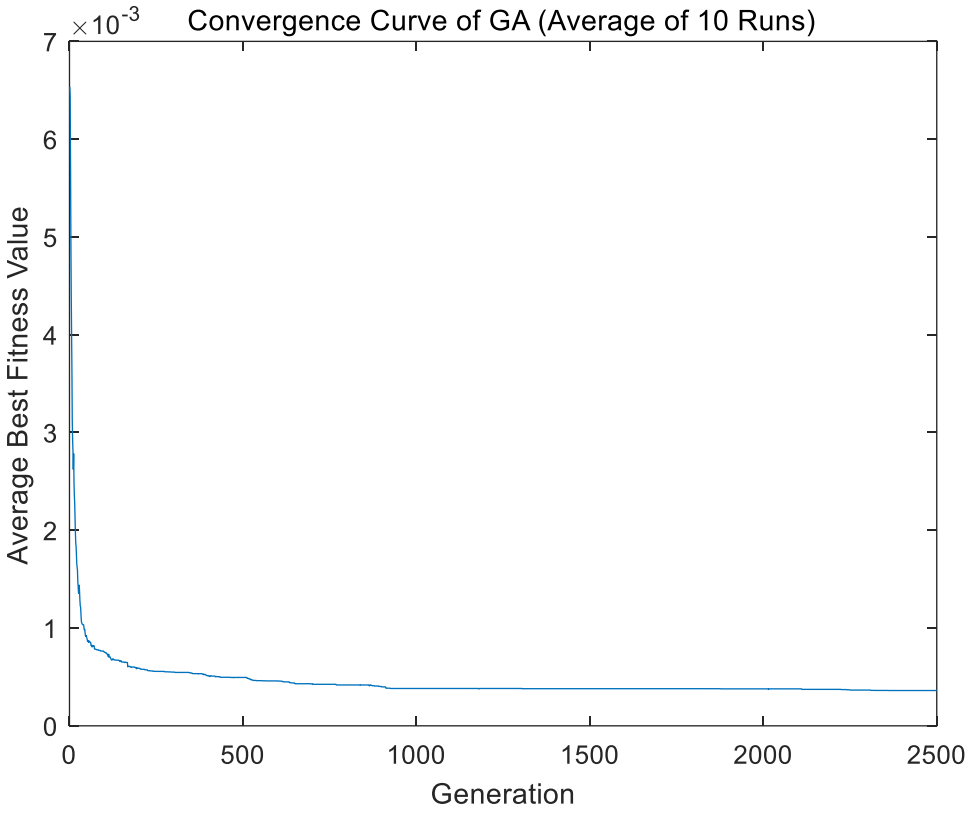


- Sixth setting

- population_size = 200;
- crossover_frac = 0.1;
- mutation_frac = 0.1;

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000341            0.0012798            0.0003611            0.0004486
```

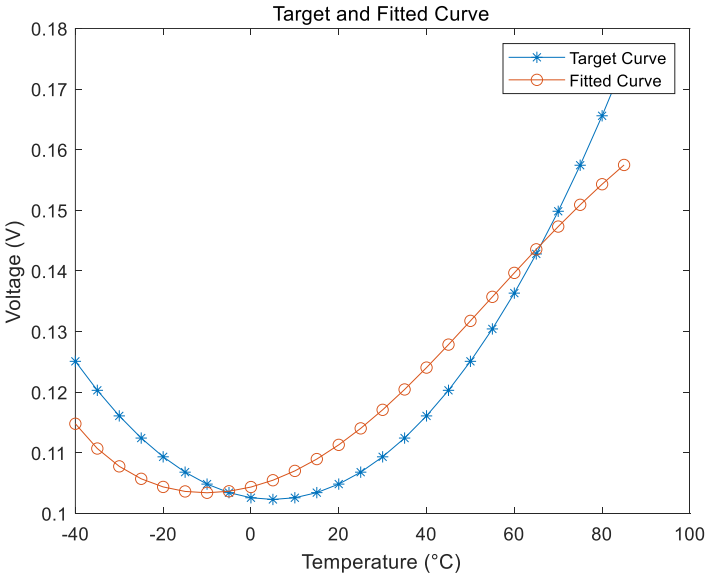


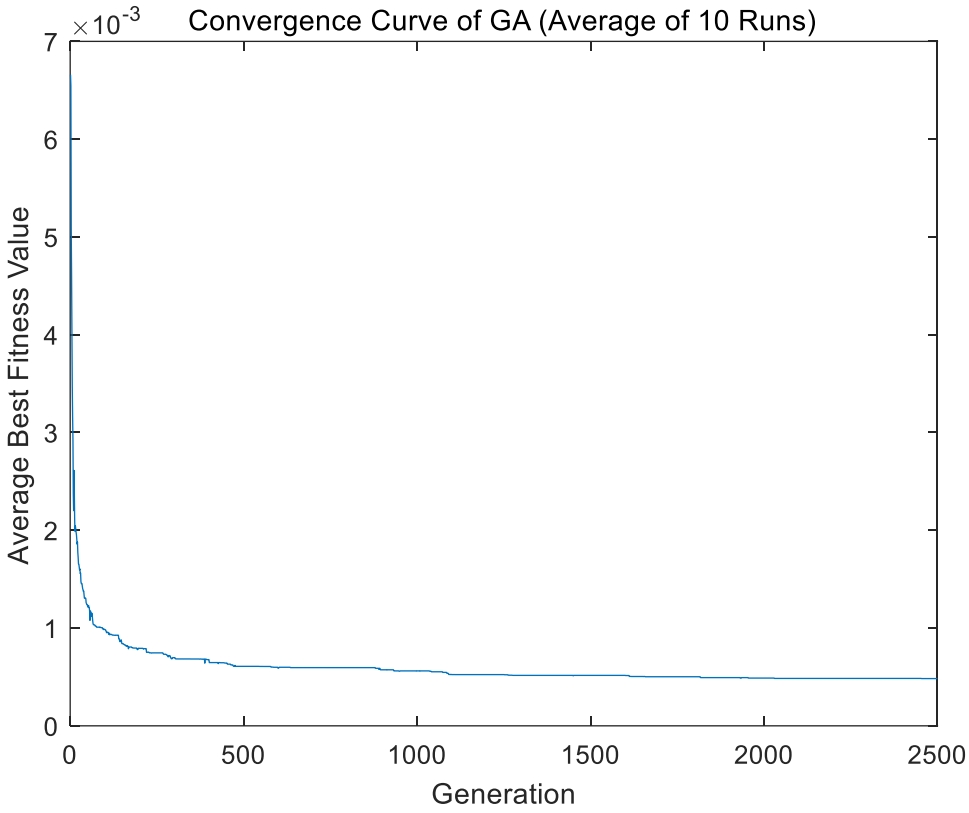


➢ Seventh setting

- `population_size = 200;`
- `crossover_frac = 0.2;`
- `mutation_frac = 0.1;`

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000668            0.0010897            0.0004832            0.0003457
```

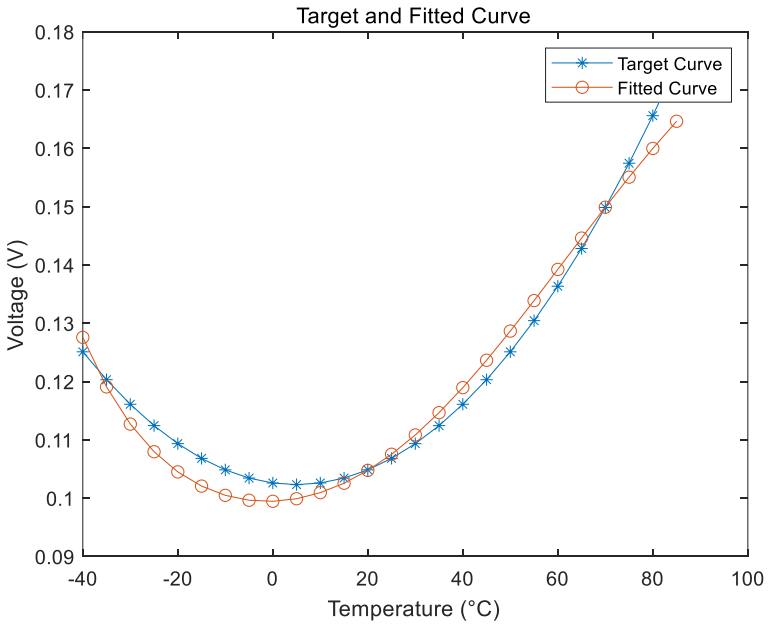


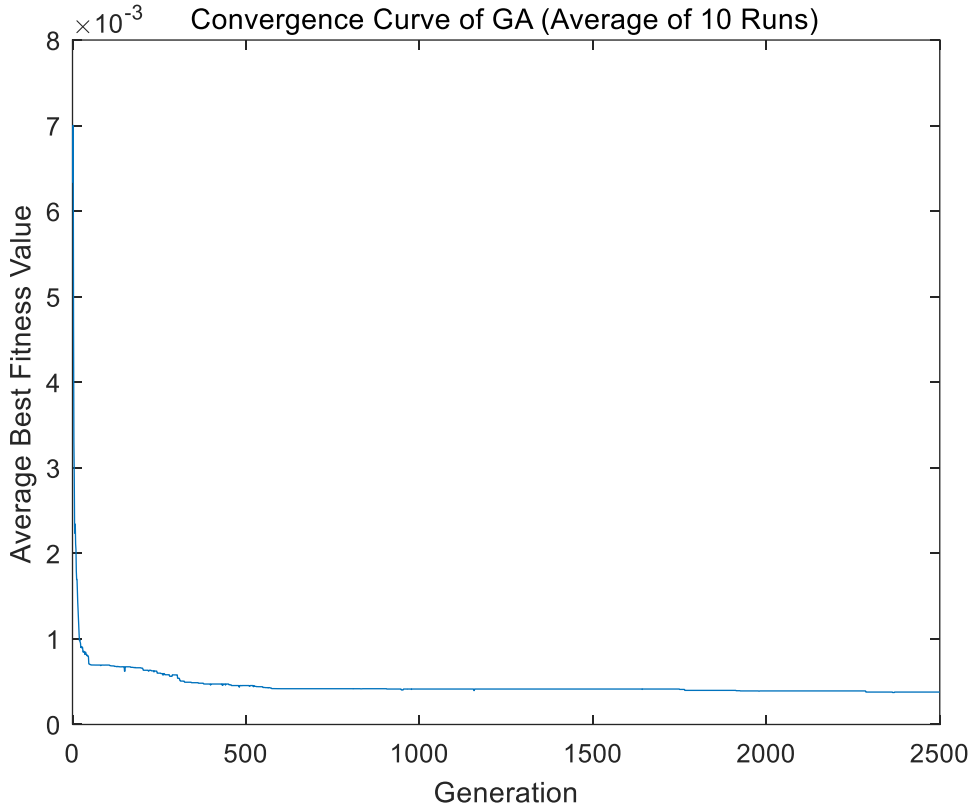


- Eighth setting

- `population_size = 200;`
- `crossover_frac = 0.4;`
- `mutation_frac = 0.1;`

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000609            0.0012930            0.0003790            0.0004567
```

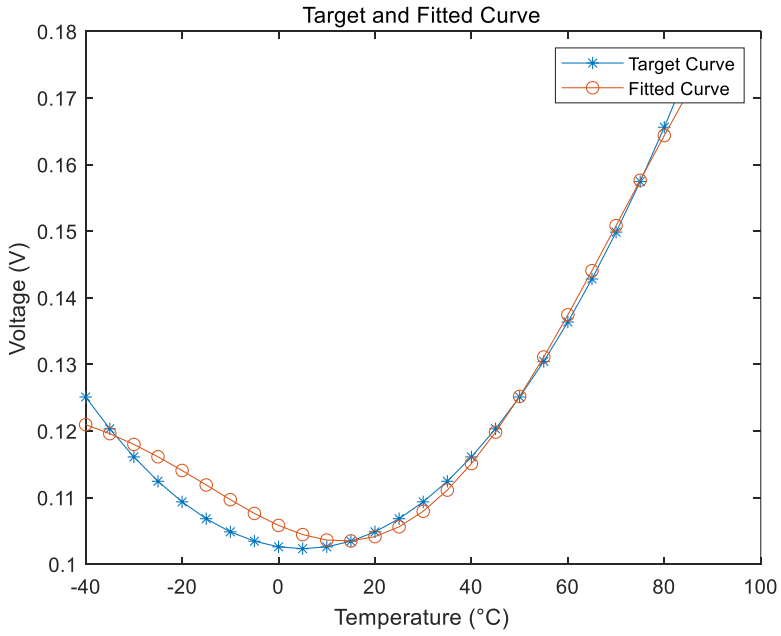


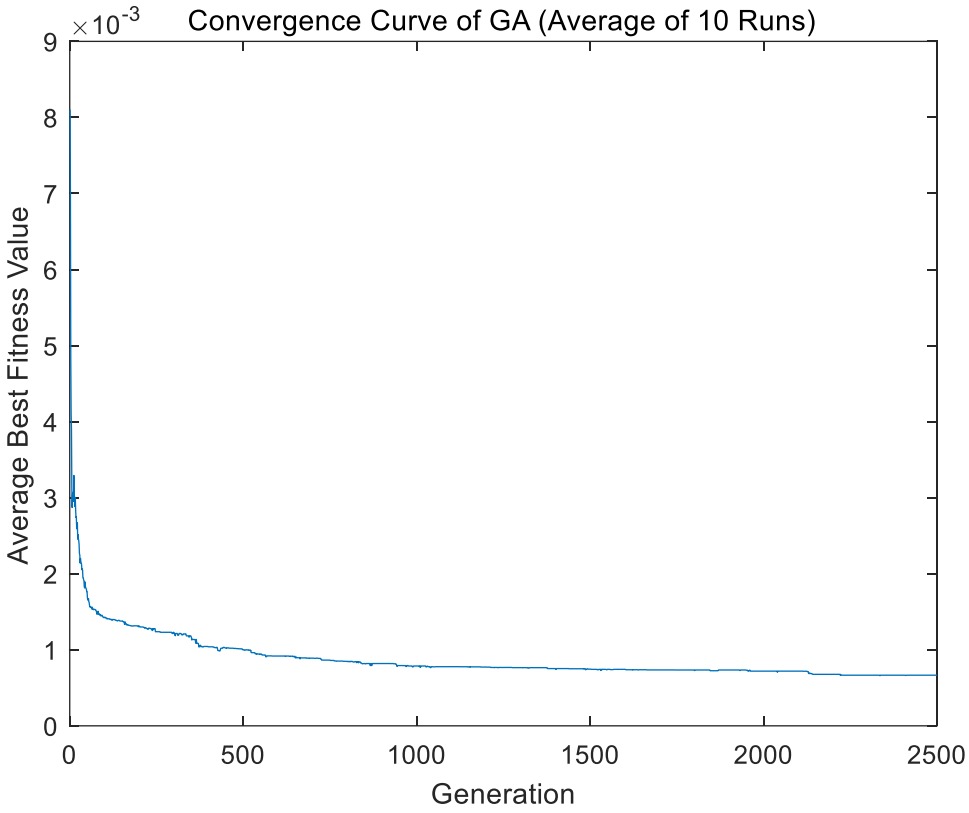


➢ Ninth setting

- population_size = 200;
- crossover_frac = 0.6;
- mutation_frac = 0.1;

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000849            0.0013227            0.0006657            0.0004770
```

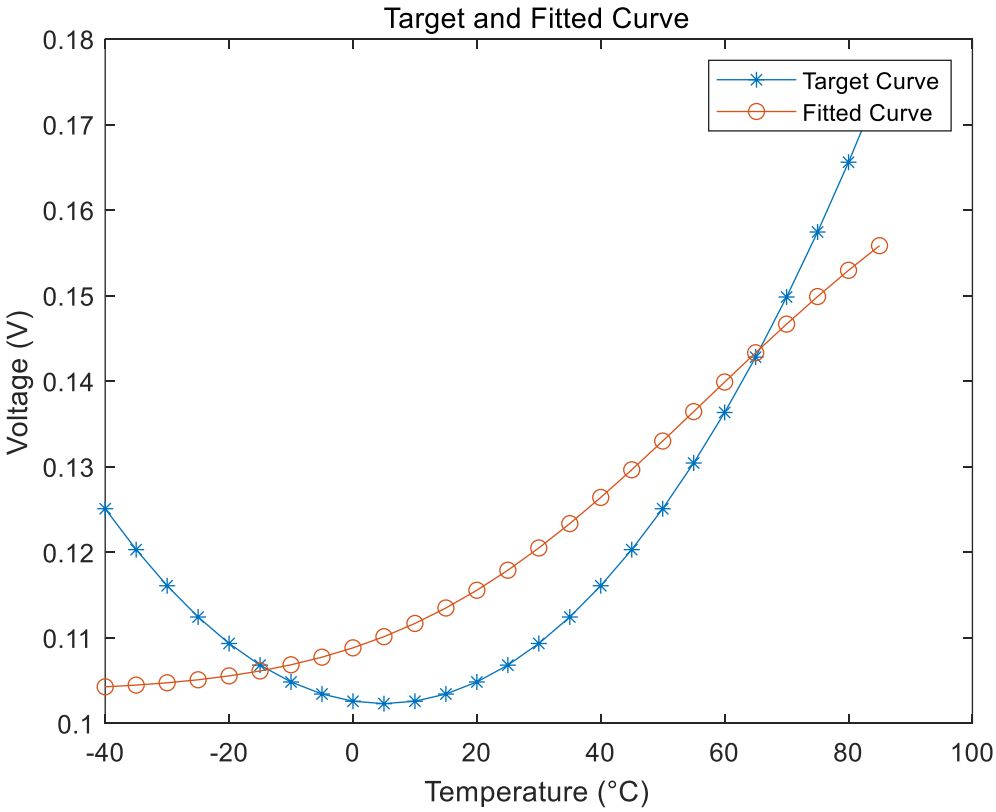


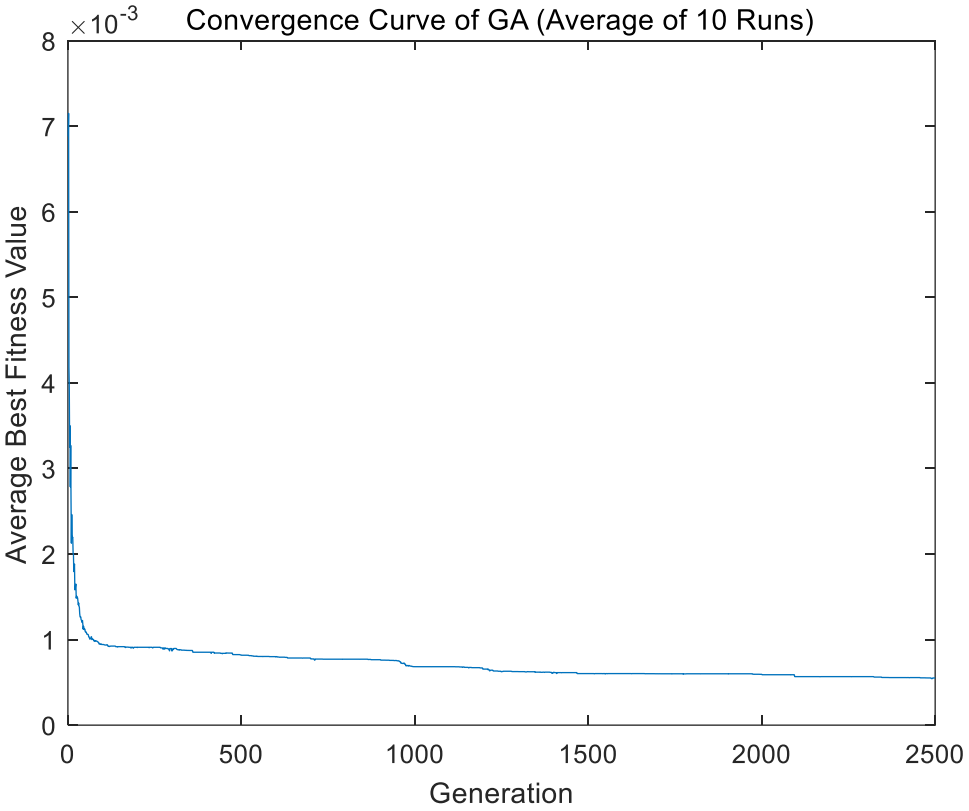


➢ Tenth setting

- population_size = 200;
- crossover_frac = 0.7;
- mutation_frac = 0.1;

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000654            0.0012930            0.0005515            0.0005225
```





Discussion

According to the above Six settings, I found two points:

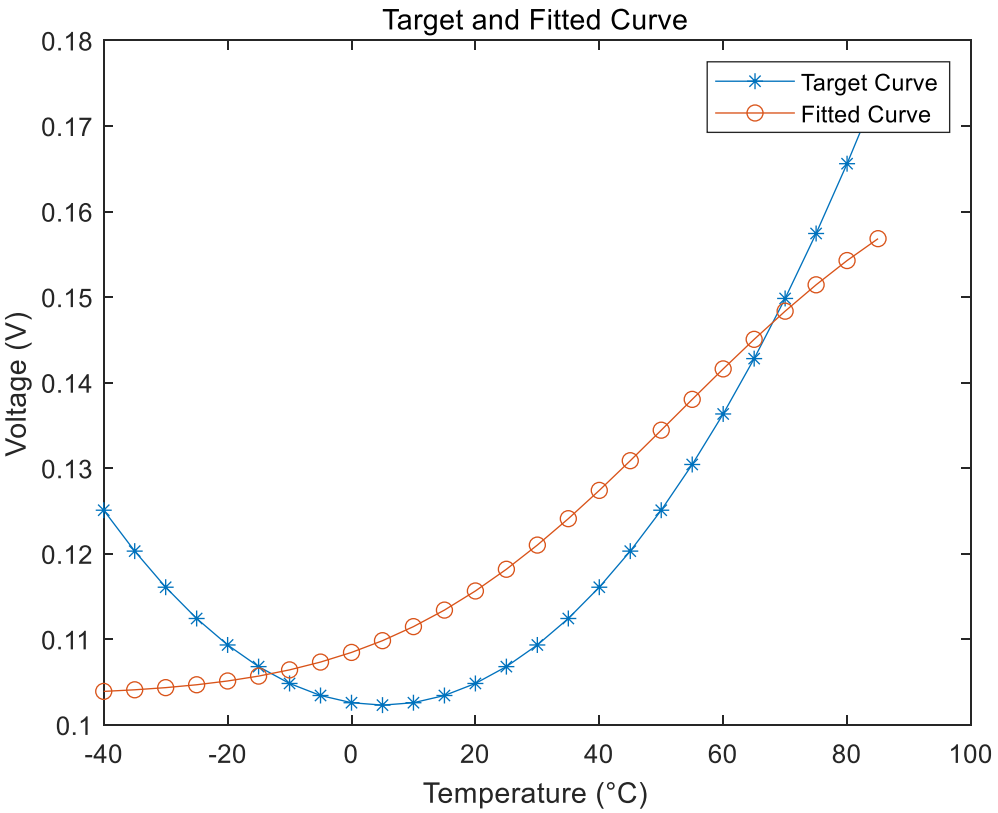
1.moderate crossover fractions (0.4–0.6) often resulted in better solutions by balancing exploration and exploitation.

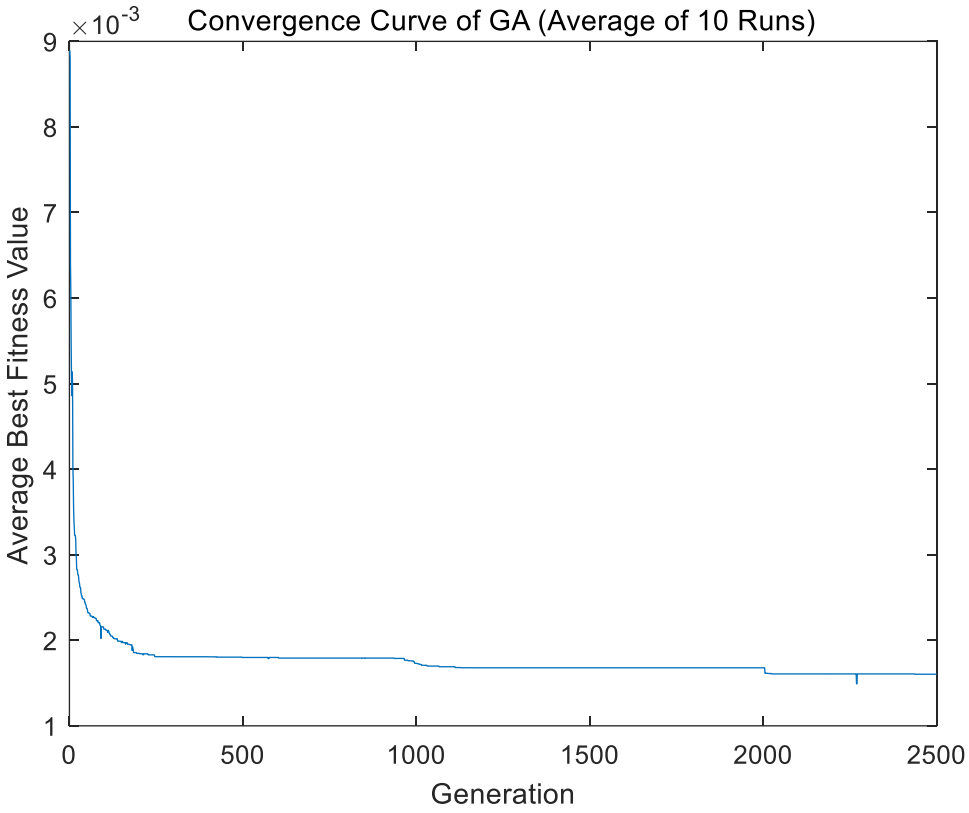
2.very high or very low crossover fractions either reduced diversity or slowed convergence.

3.mutation fraction.

- ➢ eleventh setting
- ● population_size = 200;
- ● crossover_frac = 0.1;
- ● mutation_frac = 0.01;

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0004184            0.0043076            0.0016029            0.0012356
```

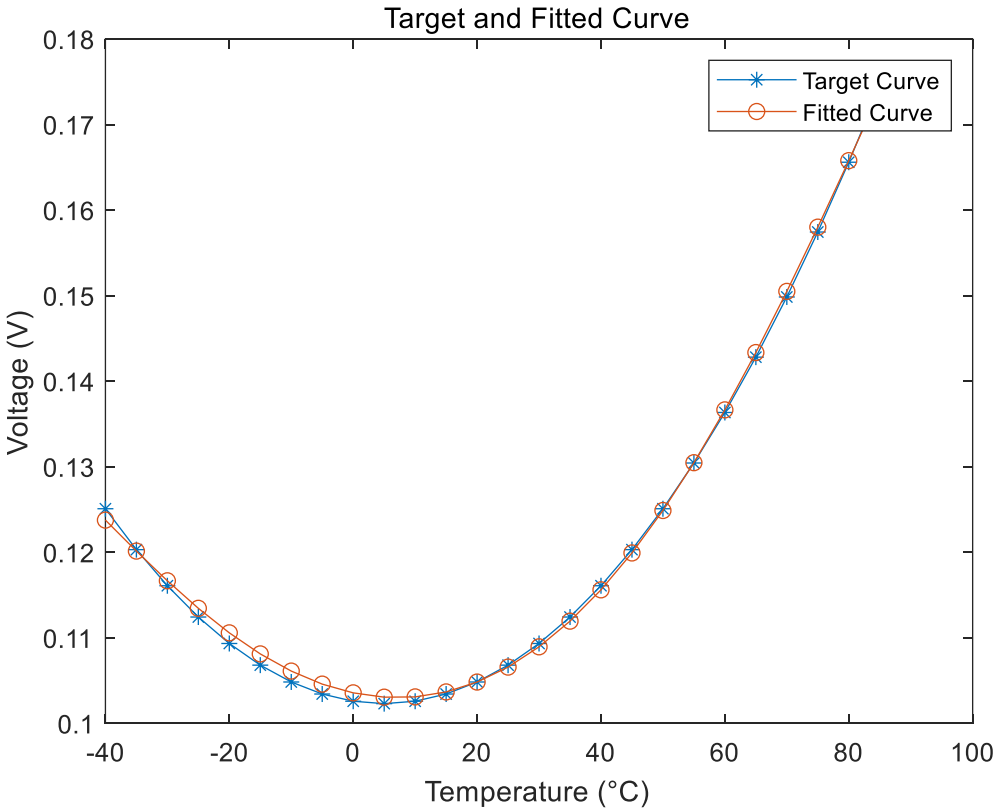


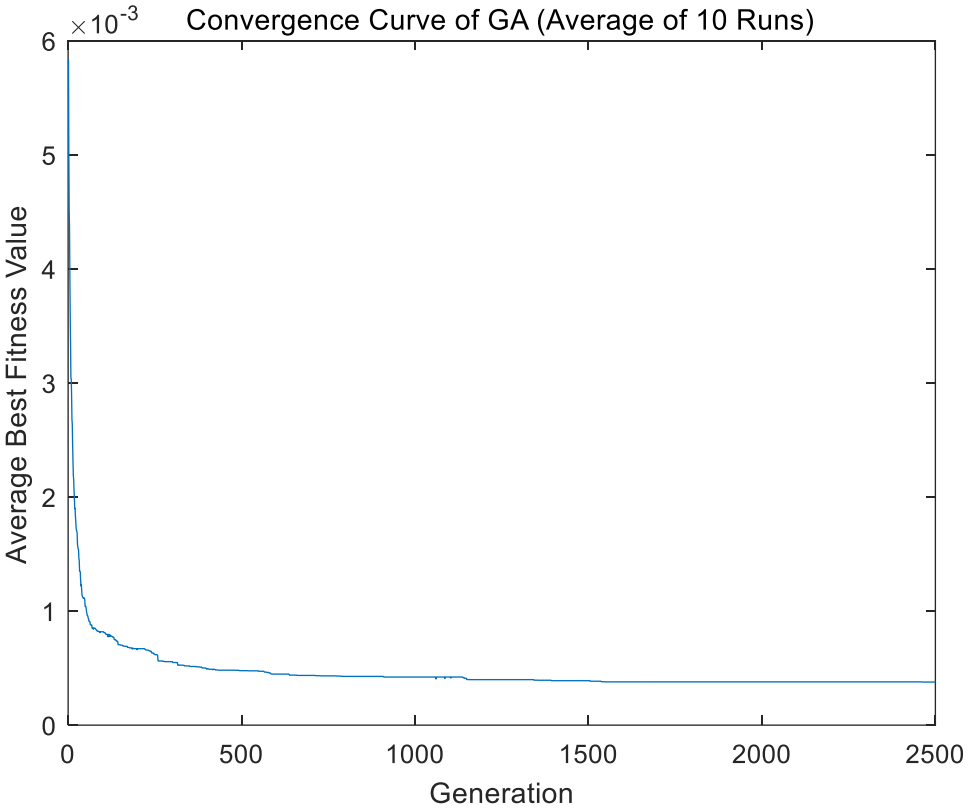


➢ Twelfth setting

- population_size = 200;
- crossover_frac = 0.1;
- mutation_frac = 0.1;

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000053            0.0010897            0.0003779            0.0004454
```

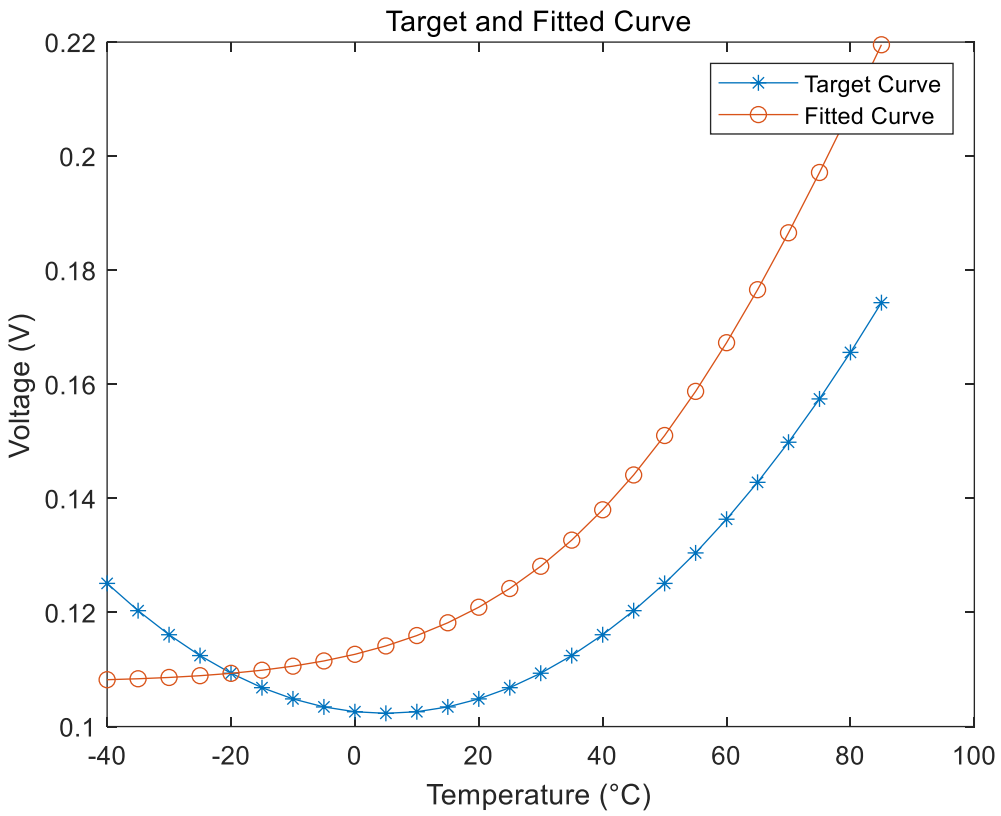


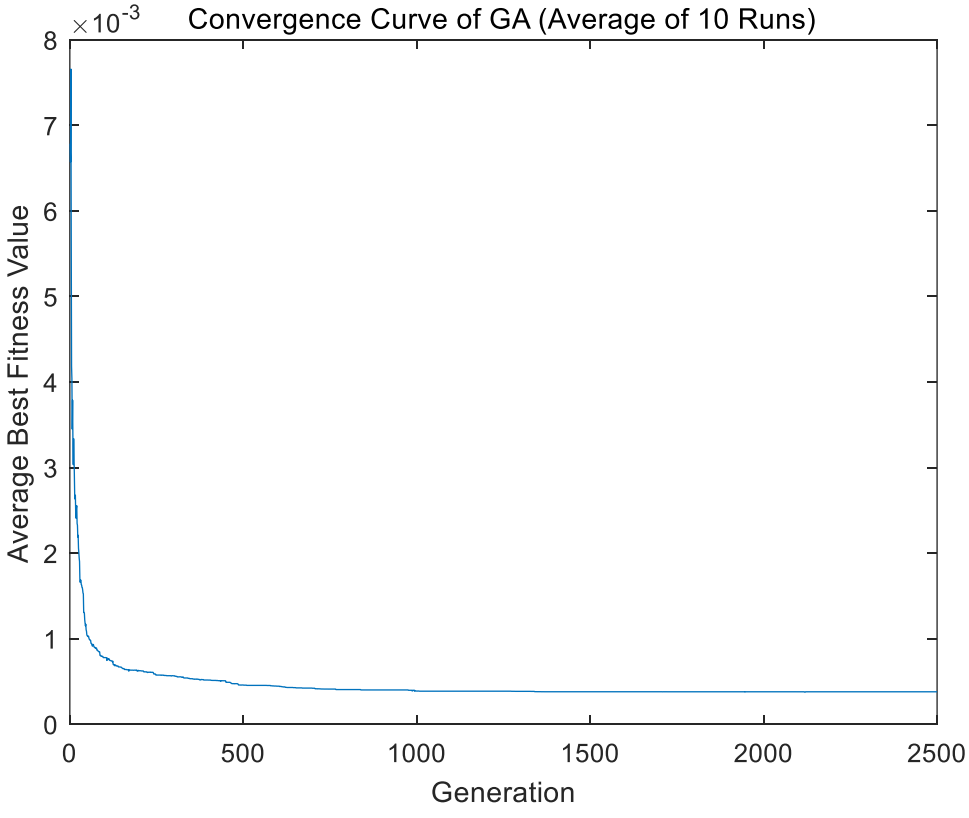


➢ Thirteenth setting

- population_size = 200;
- crossover_frac = 0.1;
- mutation_frac = 0.3;

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000357            0.0008358            0.0003791            0.0003931
```

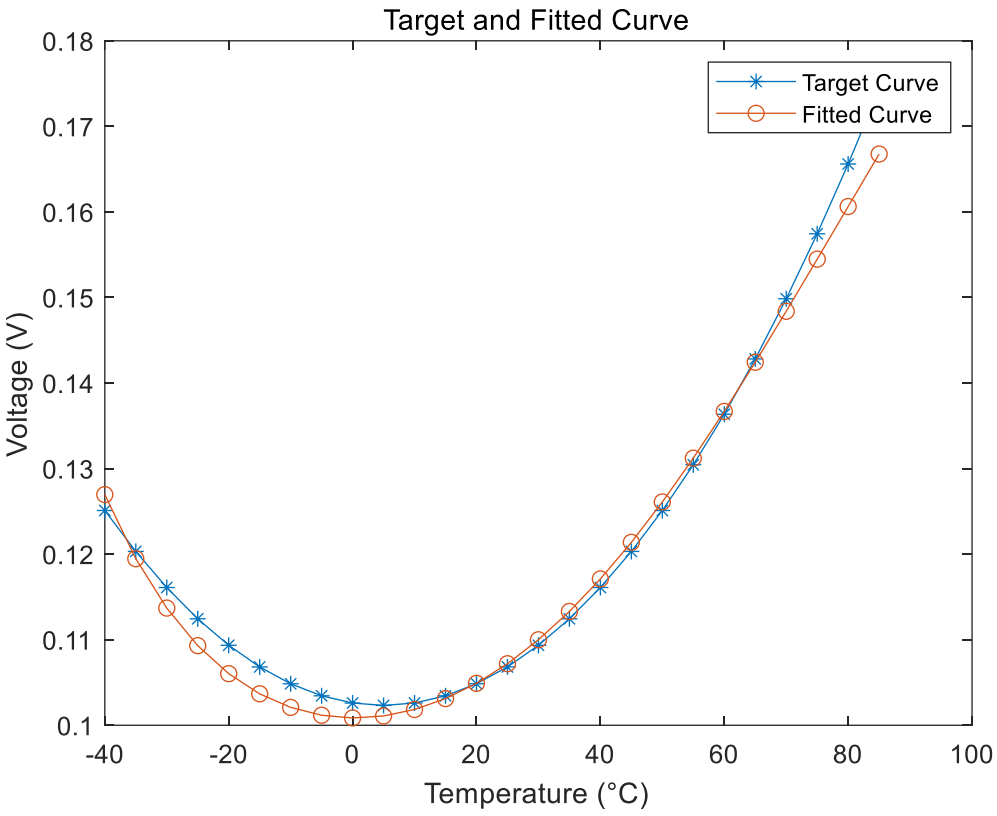


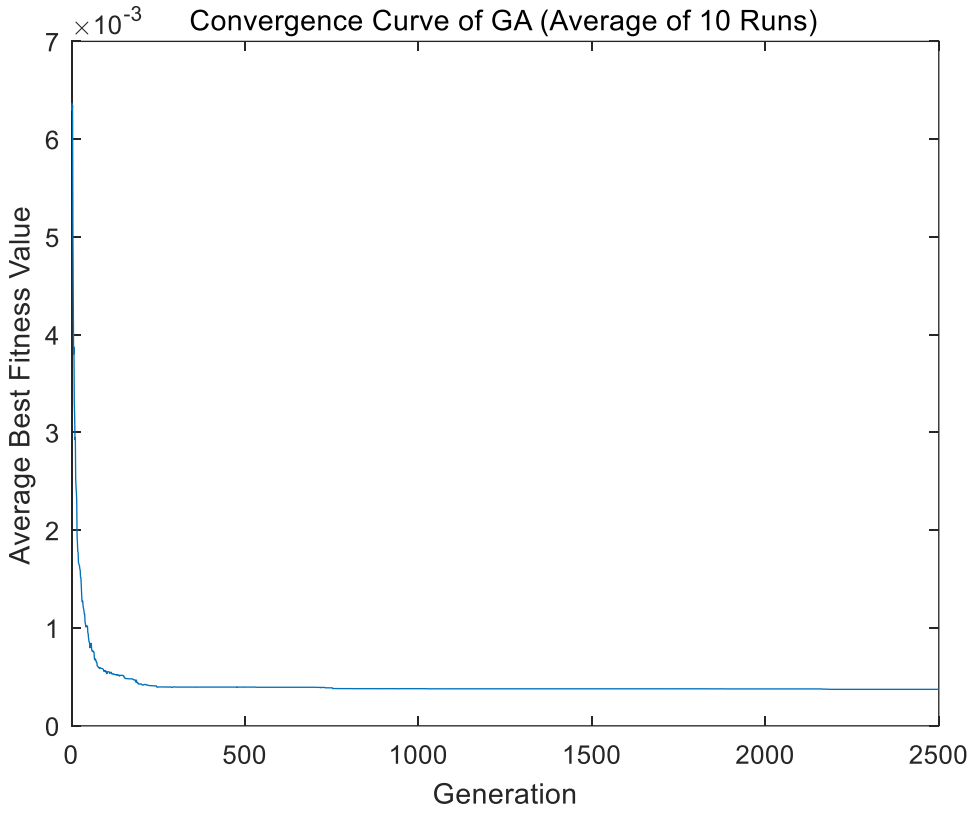


➢ Fourteenth setting

- population_size = 200;
- crossover_frac = 0.1;
- mutation_frac = 0.5;

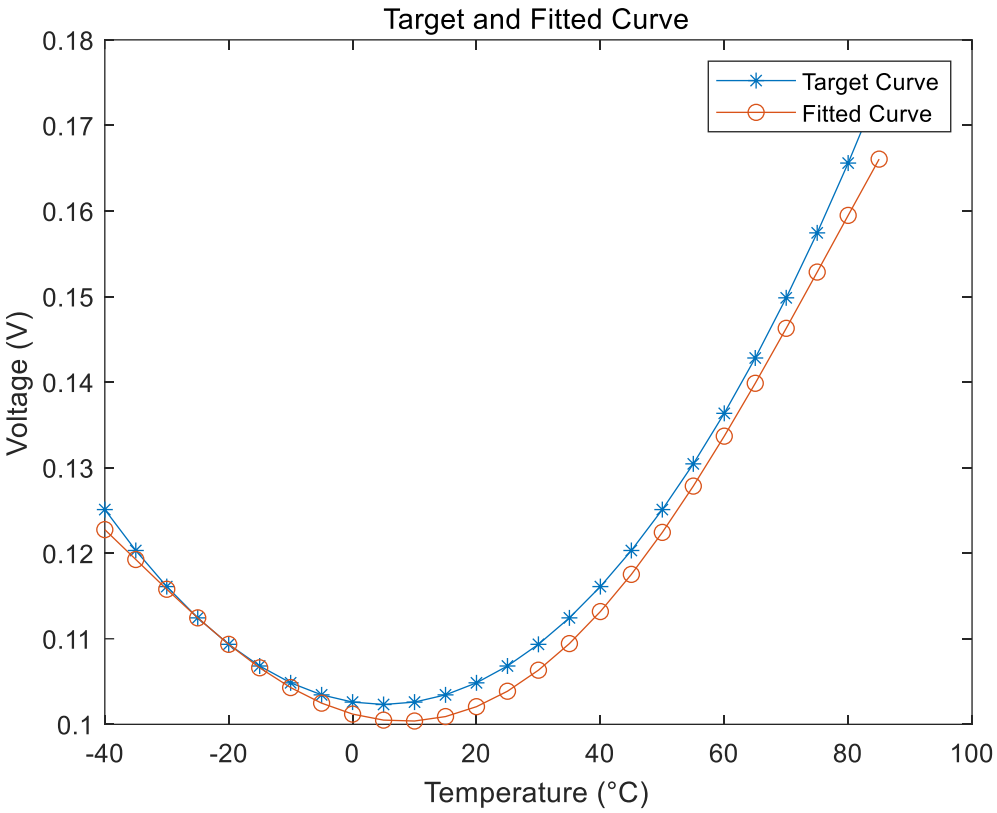
```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000053            0.0008551            0.0002888            0.0003841
>>
```

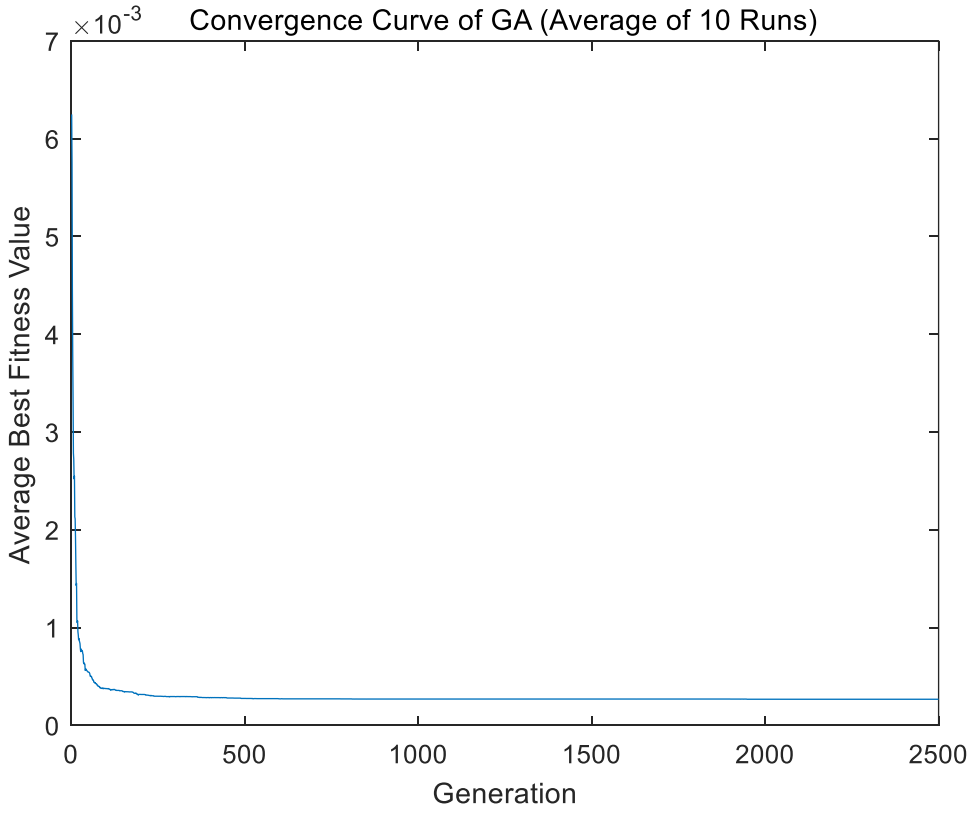




- Fifteenth setting
  - population_size = 200;
  - crossover_frac = 0.1;
  - mutation_frac = 0.7;

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000053            0.0008350            0.0002668            0.0003922
```

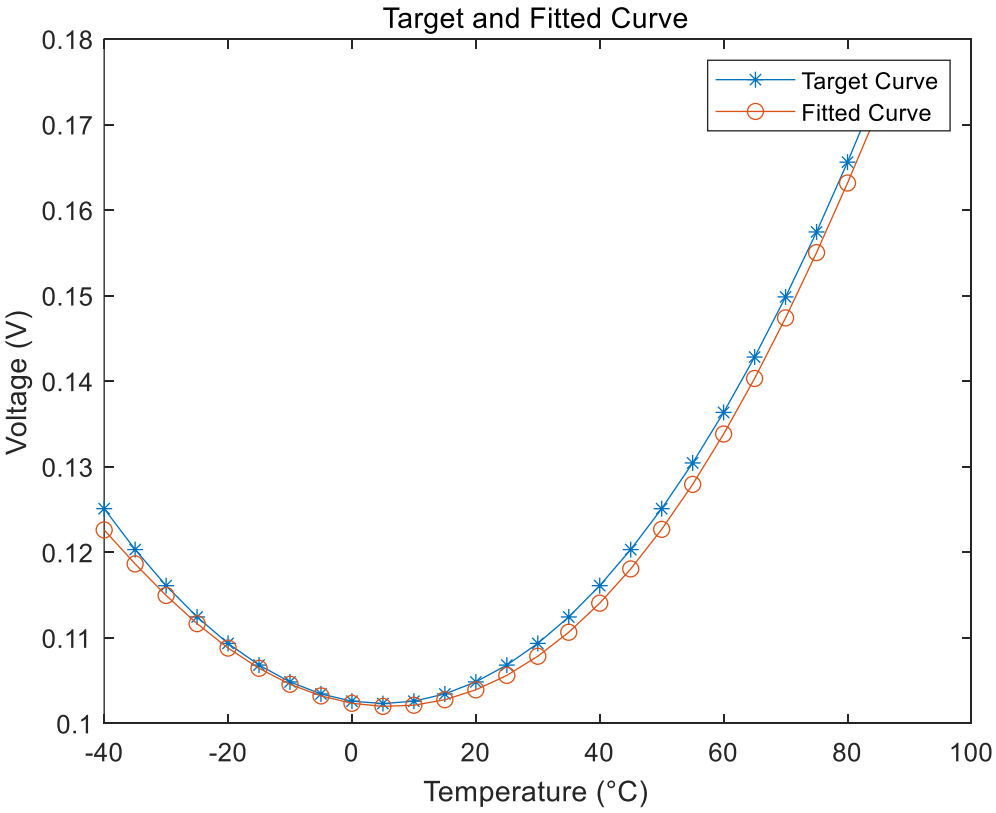


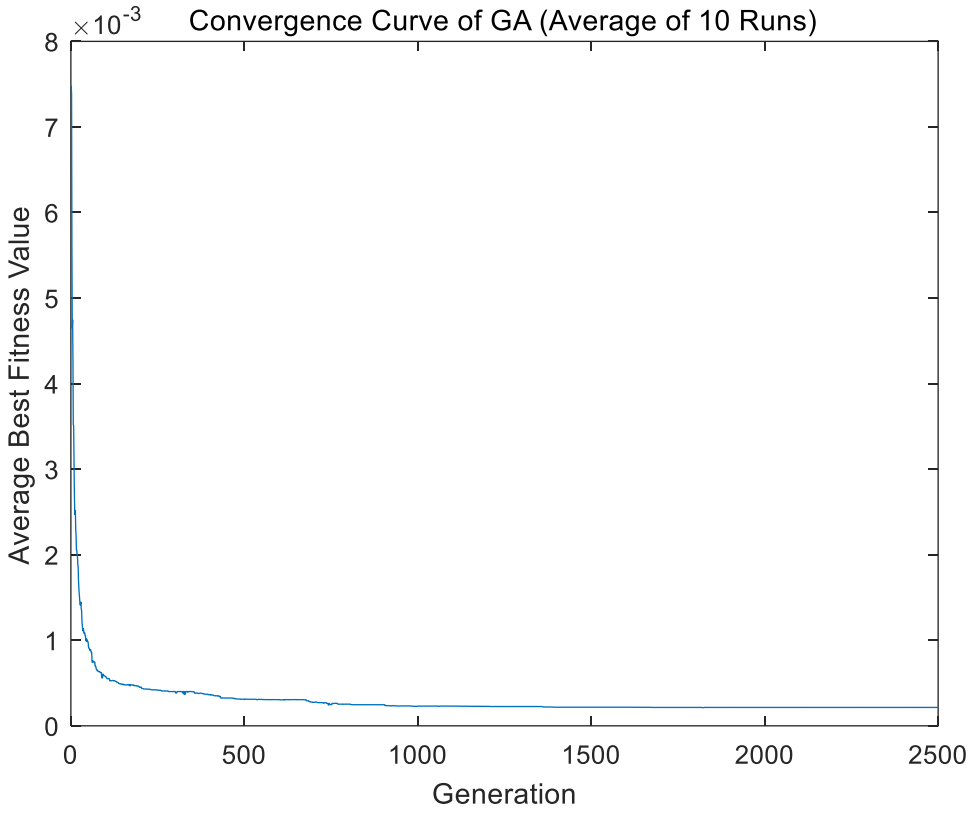


➢ Sixteenth setting

- population_size = 200;
- crossover_frac = 0.1;
- mutation_frac = 0.9;

```
>> P1
Summary of 10 Runs:
Best_Result          Worst_Result         Mean_Result          Std_Result
0.0000053            0.0008350            0.0002149            0.0003313
```





Discussion

From above 6 settings, I could find that increasing mutation fraction could improve the optimization quality, which was probably because higher mutation rates helped maintain diversity, particularly in later generations

summary

In general, population size, Crossover fraction and mutation fraction were all important for the optimization quality. Population Size and Mutation Fractions significantly affects diversity and convergence. while Crossover Fractions need careful balancing to achieve optimal exploration and exploitation.

My explanation for my design

1. Extracting My Own GA Function
   I began by reusing the Genetic Algorithm (GA) function from Lab 2 to maintain a consistent framework for optimization. The GA function was modified to handle custom parameter configurations for population size, crossover fraction, and mutation fraction. I replaced the built-in ga function in the provided RunThermistorExample script with my custom GA function.

2. Defining the Constraint Function
   To ensure the optimization adhered to specific requirements, I defined a constraint function nonlcon as follows:

```
function c = nonlcon(x)
    c = x(1) + x(2) - 100;  % Constraint: x(1) + x(2) <= 100
end
```

   This constraint ensures that the sum of the first two decision variables do not exceed 100.
   The GA incorporates this function to perform constrained optimization effectively.

3. Manually Setting GA Parameters
   I conducted experiments using 13 sets of manually chosen GA parameters, with each set involving:
   - Population Size: Values selected based on prior knowledge of its effect on optimization quality.
   - Crossover Fraction: Chosen to balance exploration and exploitation.
   - Mutation Fraction: Adjusted to maintain diversity while avoiding excessive randomness.

   This manual adjustment allowed me to systematically study the impact of these parameters on the optimization quality.

4. Executing 10 Runs for Each Parameter Setting

For each of the 13 parameter sets, the GA was executed 10 times to evaluate its robustness and performance.
For each parameter setting, I calculated:

- Best, Worst, and Mean fitness values from the 10 runs.
- Standard deviation of the fitness values across these runs.

This provided a comprehensive understanding of how parameter variations affected the optimization results.

5. Analyzing the Effect of Parameters
   After running the experiments, I analyzed the impact of the GA parameters on the optimization quality:
   the discussions were above.

6. Visualizing Results
   (a) Summary Table:
   I compiled a table summarizing the best, worst, and mean fitness values and their standard deviations for each parameter setting. This provided a clear comparison of performance across all 13 configurations.
   (b) Fitted Curve vs. Target Curve:
   Using the best solution obtained from all runs, I generated a fitted voltage curve and compared it to the target voltage curve. This demonstrated how well the GA optimized the circuit parameters.
   (c) Convergence Curve:
   For each parameter setting, I plotted the average convergence curve over 10 runs.
   This curve illustrated how quickly and consistently the GA converged to optimal solutions under different configurations.

7. Displaying Parameters and Results
   For each parameter setting, I displayed:
   - GA parameters (population size, crossover rate, and mutation rate).
   - Best, Worst, and Mean fitness values from the 10 runs.
   - Standard deviation of the fitness values across these runs.

   These results were organized into a clear table, enabling easy comparison and interpretation.